Pokhara University
Faculty of Management Studies

Course Code: CMP 276
Course Title: **Software Engineering and Project Management**
Nature of the course: Theory + Practical
Year: Second
Level: Bachelor
Semester : IV

Full marks: 100
Pass marks: 45
Credit Hrs: 3
Total periods: 48 hours
Program: BCSIT

1. Course Description

This course aims to introduce the fundamental concepts of software engineering, including software project management. It is designed to provide students with a comprehensive understanding of software engineering principles and project management practices. It intends to equip students with the knowledge and skills necessary to develop high-quality software products while effectively managing the entire software development lifecycle. At the end of this course, a student will be able to understand the fundamental concepts of software engineering and project management.

2. General Objectives

Through this course, students shall be able:

- To evaluate and relate different software processes, system models and architectural designs and assess their suitability in a given context
- To describe basic concepts and principles of requirements engineering, software implementation, testing and maintenance
- To describe the software configuration process and quality assurance
- To apply the software project management practices and principles in software development
- To understand software engineering fundamentals
- To understand programming and design practices
- To use software development tools and technologies
- To understand project management and its relevance in software development
- To understand agile principles and values
- To communicate effectively within development teams

3. Method of Instructions

General instructional techniques: lecture, discussion, readings, question answer, field work/s

Specific instructional techniques: practical works, project-based learning, self-directed learning, industry insights and case study

4. Course Contents

Specific Objectives	Contents
• Understand the fundamental nature of software and its applications • Understand the basic concept of software engineering and its processes	**Unit 1: Software and software engineering(4)** 1.1 The Nature of Software 1.2 Software Application Domains 1.3 Legacy Software 1.4 The Unique Nature of WebApps 1.5 Software Engineering 1.6 The Software Process 1.7 Software Engineering Practice 1.8 General Principles 1.9Software Myths
• Explain the use and importance of the software development life cycle. • Describe the types of software development process • Comparison of different software process models	**Unit 2: Software process models (8)** 2.1 A Generic Process Model 2.1.1 Defining a Framework Activity 2.1.2 Identifying a Task Set 2.1.3 Process Patterns 2.3 Prescriptive Process Models 2.1.1 Waterfall model and enhance waterfall model 2.2.2. Incremental Process Models 2.2.3. Rapid application development 2.2.4. Prototype and Spiral Model 2.2.5. Spiral Process Model 2.2.6. Rational Unified Process Model 2.2.7. Agile model: XP and Scrum, a toolset for the agile process 2.2.8. Big-Bang Model 2.4 Specialized Process Models 2.4.1 Component-Based Development 2.4.2 The Formal Methods Model 2.4.3 Aspect-Oriented Software Development
• Discuss software requirement • Requirement engineering • Explain about requirement management and SRS documents • Understanding and creating SRS	**Unit 3: software requirement specification and modeling (8)** 3.1 Software requirements and its types 3.2 Requirement modeling approaches 3.3 Requirement engineering 3.4 Eliciting requirements 3.5 Collaborative requirements gathering 3.6 Requirement analysis 3.7 Negotiating requirements 3.8 Requirement documentation and validation 3.9 Requirement management

	3.10 Scenario, data, Ccass and flow-oriented modeling 3.11 Requirement modeling for web apps 3.12 SRS documents and their importance

• Understand importance of software design • Discuss about software design models • Discuss different software design strategies and design patterns • Compare and contrast between Function oriented design vs. object-oriented design	**Unit 4: Design Concepts (6)** 5.1. The evolution of software design, design concepts, design processes, Design models, software design architecture 5.2. Software design strategies, interface analysis, interface design Steps, WebApp interface design, design evaluation, 5.3. Design patterns -Pattern-based software design, architectural patterns, user interface design patterns, web app design patterns 5.4. Function oriented design 5.5. Object-oriented design
• Understand software measurement and metrics • Discuss object-oriented matrices • Understand the basic concept of software quality	**Unit 5: Software measurement and metrics (2)** 6.1. Software measurement 6.2. Software metrics, Review Matrics and their use, Formal technical Reviews 6.3. Control flow graph 6.4. Cyclomatic complexity 6.5. Object-oriented matrices 6.6. Lossless decomposition **Lab Work Unit 5:** 1. Imagine you are part of a development team working on a simple web application for a bookstore. The application allows users to browse books, add them to their cart, and make a purchase. The codebase is maintained in a version control system, and you have been asked to analyze it using software metrics.
• Identify the types of software testing • Explain the software quality attributes and quality assurance • Explain quality standards	**Unit 6: Software testing and quality assurance (4)** 6.1. Software testing approach 6.2. Types of system testing (internal and external views of testing, basis path testing, control structure testing, black-box testing, white-Box testing) System debugging 6.3. Software quality attributes and quality factors 6.4. Software quality control and quality assurance Statistical software quality assurance, software reliability 6.5. Software safety 6.6. The ISO 9001 quality standards

	6.7. SEI capability maturity model 6.8. Verification and validation
• Understand the concept of configuration management. • Discuss types of software maintenance. • Understand Software maintenance costs.	**Unit 7: Configuration management and software maintenance (3)** 7.1. Software configuration and change management 7.3. Version and release management 7.5. Types of software maintenance 7.4. Need for software maintenance 7.6. Software maintenance process models 7.7. Software maintenance cost
• Explain software project management and planning. • Discuss project estimation techniques. • Understand the COCOMO model.	**Unit 8: Software project management (4)** 8.1. Software project, project manager, qualities of project manager 8.2. Activities in project management 8.3. Software project planning and management plan 8.5. Software project scheduling and techniques 8.6. Software project team management and organization 8.7. Project estimation techniques: COCOMO model
• Understand what is project scheduling and identify and use project scheduling tools	**Unit 9: Project Scheduling (2)** 9.1. Project scheduling 9.2. Defining a task for the software project 9.3. Project scheduling and its types 9.4. Tools for project scheduling 9.5. PERT and Scheduling CPM
• Discuss risk, software configuration management	**Unit 10: Risk Management (2)** 10.1. Software risk 10.2. Risk Identification, Projection and Refinement 10.3. Risk Projection 10.4. Risk Refinement 10.5. Risk analysis management and process 10.6.The RMMM plan
• Discuss about the basic concept of software re-engineering, forward and reverse engineering • Discuss software reuse	**Unit 11: Concept of software re-engineering (3)** 11.1. Steps in re-engineering 11.2. Re-engineering process and process models 11.3. Forward engineering, Reverse engineering process 11.4. Characteristics of forward and reverse engineering 11.5. Restructuring 11.6. The economy of engineering 11.7. Software reuse 11.8. Economic of reengineering

	Unit 12: Emerging trends in software engineering (2) 12.1. Observing Software Engineering Trends 12.2. Identifying "Soft Trends" 12.3. Technology Directions 12.4. Tools related trends 12.5. Software engineer's responsibility

5. Laboratory Work

In this laboratory session on Software Engineering and Project Management, students engage in practical exercises to apply key principles of software development and project management methodologies. Students will gain hands-on experience in designing, testing, and debugging software, emphasizing the importance of efficient project planning and execution. The lab integrates tools and techniques for version control, collaboration, and issue tracking. Through simulated project scenarios, students learn to manage resources, timelines, and deliverables, fostering skills essential for successful software development projects. The listed units in this subject cultivate a comprehensive understanding of software engineering practices and project management essentials, preparing students for real-world applications in the field.

Some important contents that should be included in lab exercises are as follows:

Units	Laboratory Activity
Unit 1: Software and software engineering	- Identify the different types of software available in the market, list their features, and identify the challenges associated with the system.
Unit 2: Software process models	- Visit any two software development companies and identify the process models used.
Unit 3: software requirement specification and modeling	- Create a detailed Software Requirement Specification document for an Online Bookstore System, outlining the functional and non-functional requirements. - Create a SRS sample of any software you are developing in this semester as a part of your project work.
Unit 4: Design Concepts	- Create the design solution to the problems identified in Lab work of unit I.
Unit 5: Software measurement and metrics	- Imagine you are part of a development team working on a simple web application for a bookstore. The application allows users to browse books, add them to their cart, and make a purchase. The codebase is maintained in a version control system, and you have been asked to analyze it using software metrics. **Tools Required:**

	- Integrated Development Environment (IDE) - Version Control System (e.g., Git). - Code analysis tools (e.g., SonarQube, PMD, or any tool of your choice).
Unit 6: Software testing and quality assurance	- Create a basic Quality Assurance Plan outline for a software development project. - Given a hypothetical software project scenario, propose three metrics that could assist project managers in making informed decisions.
Unit 7: Configuration Management and Software Maintenance	- Use any version control software.
Unit 8: Software project management	- AgileTrack - Implementing Agile Methodology for Project Management. - Use any project management tools to manage the project created in Unit 3 or Project II.
Unit 9: Project Scheduling	- Create a complete project schedule using any project scheduling tools.
Unit 10: Risk Management	- As a part of the group discussion take any software, identify risks and make an RMMM plan.
Unit 11: Concept of software re-engineering	- Identify different versions of popular (system software and application software) and identify the reused component.
Unit 12: Emerging trends in software engineering	- Identify and trending software and present the features in class.

Note:

1. Motivate students to create small project work integrating all of the above concepts.
2. Each of the above lab sessions should cover more than 3 hours of practical work.

6. Evaluation system and student's responsibility

In addition to the formal exam(s), the internal evaluation of a student may consist of quizzes, assignments, lab reports, projects, class participation, etc. The tabular presentation of the internal evaluation is as follows.

External Evaluation	**Marks**	**Internal Evaluation**	**Marks**	
Semester End Examination		**Theory**		40
		Practical		20
	40	Attendance & Class Participation	5	
		Lab Report/Project Report	5	

		Practical Exam/Project Work	5	
		Viva	5	
Total External	**40**	**Total Internal**		**60**
Full Marks: 40 + 40+ 20 = 100				

Student's Requirements

Each student must secure at least 45% marks separately in both internal assessment and practical evaluation with 80% attendance in the class to appear in the Semester End Examination. Failing to get such a score will be given NOT QUALIFIED (NQ) to appear in the semester-end Examinations. Students are advised to attend all the classes, formal exams, tests, etc. and complete all the assignments within the specified period. ***Students are required to complete all the requirements defined for the completion of the course.***

7. Prescribed Books and References

Prescribed Text Books:

1. Roger S. Pressman, Software Engineering (7th Edition). Boston, Mass: McGraw Hill.

Reference Books:

2. Sommervill, I. (2011). *Software engineering* (9th ed.). Boston: Pearson.
3. Bob Hughes and Mike Cotterell (Latest Edition). Software Project Management (Latest Edition). Boston, Mass: McGraw Hill.
4. Pressman, R. S. (2010). *Software engineering: a practitioner's approach* (7th ed.). Boston, Mass: McGraw Hill.
5. Software engineering, Udit Agarwal
6. Software Engineering Fundamentals, " Ali Behforooz and Frederick J. Hudson